

## **Trabajo Práctico Integrador**

### **Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas**

#### **Alumnos**

Cardenas Francisco

Caliari Sofia

**Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.**

#### **Programación I**

**Junio 2026**

# Índice

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1. INTRODUCCIÓN .....</b>                      | <b>3</b>  |
| <b>2. MARCO TEÓRICO.....</b>                      | <b>3</b>  |
| <b>3. DESCRIPCIÓN DEL PROBLEMA .....</b>          | <b>7</b>  |
| <b>4. DECISIONES TÉCNICAS Y ARQUITECTURA.....</b> | <b>7</b>  |
| <b>5. ESTRUCTURAS DE DATOS UTILIZADAS.....</b>    | <b>7</b>  |
| <b>6. FUNCIONALIDADES IMPLEMENTADAS .....</b>     | <b>8</b>  |
| <b>7. DIFICULTADES ENCONTRADAS.....</b>           | <b>9</b>  |
| <b>8. CONCLUSIONES .....</b>                      | <b>9</b>  |
| <b>9. BIBLIOGRAFIA Y WEBGRAFÍA .....</b>          | <b>9</b>  |
| <b>10. ENLACES DEL PROYECTO .....</b>             | <b>10</b> |

## **1. INTRODUCCIÓN**

El presente trabajo práctico tiene como objetivo desarrollar una aplicación en Python para la gestión de información de países mediante el uso de listas, diccionarios, funciones y archivos CSV.

El sistema permite almacenar, consultar, actualizar, ordenar y analizar datos relacionados con distintos países, aplicando los conocimientos adquiridos durante la cursada de Programación 1.

El desarrollo fue realizado de manera colaborativa entre ambos integrantes del equipo, utilizando Git y GitHub como herramientas de control de versiones para organizar el trabajo, integrar los avances realizados por cada miembro y mantener una versión unificada del proyecto.

Además de resolver el problema planteado, el proyecto permitió reforzar conceptos fundamentales de programación, mejorar las habilidades de trabajo en equipo y adquirir experiencia en el uso de herramientas utilizadas habitualmente en entornos de desarrollo profesional.

## **2. MARCO TEÓRICO**

### **LISTAS**

Una lista es una estructura de datos que permite almacenar una secuencia ordenada de elementos, los cuales pueden ser de cualquier tipo: números, cadenas, booleanos, incluso otras listas. A diferencia de otros lenguajes que requieren declarar el tipo de dato, en Python una lista puede contener elementos de distintos tipos.

Las listas son mutables, lo que significa que pueden modificarse después de ser creadas: se pueden agregar, eliminar o cambiar elementos.

En este trabajo utilizamos una lista para almacenar todos los países cargados desde el archivo CSV. Cada elemento de la lista representa un país, permitiendo agregar nuevos registros, realizar búsquedas, aplicar filtros y efectuar ordenamientos sobre el conjunto de datos.

### **DICCIONARIOS**

Un diccionario es una estructura de datos que permite almacenar información mediante un sistema de pares clave-valor. Es decir, cada elemento que contiene tiene una clave (key) única, asociada a un valor (value). A este tipo de estructura también se lo conoce como tabla de dispersión o mapa asociativo en otros lenguajes de programación.

Decidimos representar cada país mediante un diccionario porque cada registro posee distintos atributos relacionados entre sí: nombre, población, superficie y continente. Gracias a esta estructura pudimos acceder fácilmente a cada dato utilizando claves descriptivas, por ejemplo:

```
pais["nombre"]
```

```
pais["poblacion"]
```

```
pais["superficie"]
```

```
pais["continente"]
```

Esto hizo que el código fuera más claro y legible que si se hubieran utilizado listas simples con posiciones numéricas.

## **ESTRUCTURAS CONDICIONALES**

En programación, una estructura condicional permite ejecutar diferentes bloques de código según si una condición se cumple o no. Esto es fundamental para la toma de decisiones en un programa.

Python usa la palabra clave `if` para definir una condición, y opcionalmente se pueden agregar `else` y `elif` para manejar distintos casos. Por medio de ellas es posible que el programa defina qué bloques de código ejecutar primero o directamente no hacerlo.

Durante el desarrollo se utilizaron sentencias `if`, `elif` y `else` para validar los datos ingresados por el usuario, verificar si un país existía antes de actualizarlo, controlar rangos mínimos y máximos en los filtros y mostrar mensajes de error cuando correspondía.

Por ejemplo, al filtrar por población se verificó que el valor mínimo no fuera mayor que el máximo antes de realizar la búsqueda.

## **ESTRUCTURAS REPETITIVAS**

Las estructuras repetitivas permiten ejecutar instrucciones varias veces sin necesidad de escribir el mismo código repetidamente.

En el proyecto utilizamos ciclos `for` para recorrer la lista de países al momento de buscar, filtrar, ordenar o calcular estadísticas. También se emplearon ciclos `while` para solicitar nuevamente los datos cuando el usuario ingresaba información inválida.

Estas estructuras permitieron automatizar tareas y evitar repeticiones innecesarias de código.

## **FUNCIONES**

Las funciones permiten encapsular tareas específicas dentro de bloques reutilizables de código.

Cada una de las operaciones del sistema fue implementada mediante funciones independientes, como por ejemplo:

agregar\_paises()

buscar\_pais()

actualizar\_pais()

filtrar\_poblacion()

ordenar\_nombre()

promedio\_poblacion()

Esta organización permitió mejorar la legibilidad del programa y facilitar el trabajo colaborativo entre los integrantes.

### **ARCHIVOS CSV**

Un archivo CSV (Comma Separated Values o Valores Separados por Comas) es un tipo de archivo de texto plano diseñado para almacenar datos en forma de tabla. Aunque al abrirlo con un editor de notas parece simple texto, su estructura interna sigue reglas muy precisas que permiten organizar grandes volúmenes de información de manera eficiente.

En este proyecto se utilizó el archivo paises.csv como fuente de información inicial. Mediante el módulo csv de Python se cargaron los registros en memoria para luego procesarlos utilizando listas y diccionarios. La utilización de archivos CSV permitió separar los datos del código y trabajar con información persistente de manera sencilla.

### **ORDENAMIENTO Y FILTRADO**

El ordenamiento consiste en organizar los datos según un criterio determinado, mientras que el filtrado permite mostrar únicamente los registros que cumplen una condición.

Estas técnicas fueron aplicadas para ordenar los países por nombre, población o superficie, así como también para filtrar registros por continente o por rangos de población y superficie.

Estas operaciones permiten mejorar el análisis de la información y facilitar la consulta de los datos almacenados.

### **ESTADÍSTICAS**

Las estadísticas descriptivas permiten resumir información relevante a partir de un conjunto de datos.

En el sistema se implementaron funciones capaces de identificar el país con mayor población, el país con menor población, calcular promedios y determinar la cantidad de países pertenecientes a cada continente.

Estas funcionalidades permiten obtener información significativa a partir de los datos almacenados en el sistema.

## **VALIDACIÓN Y MANEJO DE ERRORES**

La validación de datos es fundamental para garantizar la calidad y consistencia de la información procesada.

Durante el desarrollo se implementaron controles para evitar campos vacíos, impedir el ingreso de números negativos, verificar formatos correctos y manejar posibles errores durante la lectura del archivo CSV.

Estas validaciones mejoran la robustez del programa y evitan resultados incorrectos causados por datos inválidos.

## **GIT Y GITHUB**

Git es el sistema de control de versiones que corre localmente en tu máquina. GitHub es la plataforma en la nube que almacena tus proyectos (repositorios remotos) y facilita la colaboración.

Git permitió registrar los cambios realizados en el código a lo largo del proyecto mediante commits, facilitando el seguimiento de las modificaciones y la recuperación de versiones anteriores cuando fue necesario.

Por su parte, GitHub fue utilizado como repositorio remoto para almacenar el proyecto y compartirlo entre los integrantes del equipo. Esto permitió que cada miembro trabajara en distintas funcionalidades de manera independiente y luego integrara sus aportes al proyecto principal.

Durante el desarrollo se utilizaron comandos como:

- git add para preparar los archivos modificados.
- git commit para registrar cambios en el historial del proyecto.
- git pull para obtener actualizaciones realizadas por otros integrantes.
- git push para enviar los cambios al repositorio remoto.

### **3. DESCRIPCIÓN DEL PROBLEMA**

Se desarrolló un sistema para gestionar información de distintos países a partir de datos almacenados en un archivo CSV. El programa permite agregar nuevos países, actualizar información existente, realizar búsquedas, aplicar filtros, ordenar registros y obtener estadísticas relevantes.

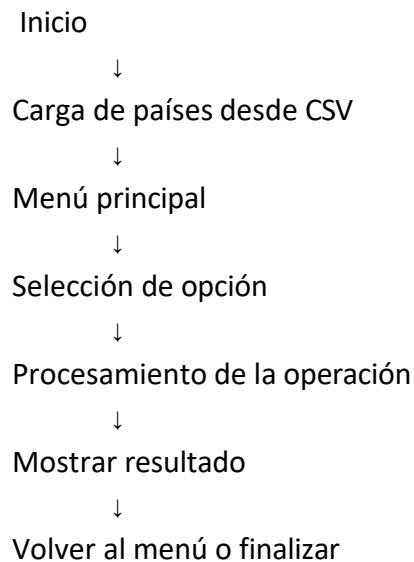
Para el manejo de los datos se utilizaron estructuras de datos dinámicas, lo que permitió almacenar y procesar la información de forma flexible y eficiente, facilitando las distintas operaciones implementadas en el sistema.

### **4. DECISIONES TÉCNICAS Y ARQUITECTURA**

Para el desarrollo se optó por una estructura modular basada en funciones, donde cada función tiene una única responsabilidad.

Esta decisión permitió facilitar el mantenimiento del código, reducir la repetición de lógica, mejorar la legibilidad y favorecer el trabajo colaborativo entre los integrantes.

#### **Diagrama de Flujo General**



### **5. ESTRUCTURAS DE DATOS UTILIZADAS**

#### **LISTA PRINCIPAL**

La información de todos los países se almacena dentro de una lista.

```
países = []
```

### DICCIONARIO DE PAÍS

Cada elemento de la lista corresponde a un país.

```
{  
    "nombre": "",  
    "poblacion": 0,  
    "superficie": 0,  
    "continente": ""  
}
```

### DICCIONARIO AUXILIAR PARA ESTADÍSTICAS

Para calcular la cantidad de países por continente se utilizó un diccionario de conteo.

```
continentes = {}
```

## **6. FUNCIONALIDADES IMPLEMENTADAS**

### GESTIÓN DE PAÍSES

- Agregar país.
- Actualizar población y superficie.
- Buscar país por nombre.

### FILTROS

- Filtrado por continente.
- Filtrado por rango de población.
- Filtrado por rango de superficie.

### ORDENAMIENTOS

- Ordenamiento por nombre.
- Ordenamiento por población.
- Ordenamiento por superficie.

### ESTADÍSTICAS

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

### VALIDACIONES

- Campos vacíos.



- Valores numéricos inválidos.
- Valores negativos.
- Rangos inconsistentes.
- Errores de lectura en el archivo CSV.

## **7. DIFICULTADES ENCONTRADAS**

Uno de los principales desafíos del proyecto fue la integración del trabajo realizado por ambos integrantes mediante Git y GitHub. Durante el desarrollo surgieron diferencias entre las versiones locales y remotas del repositorio, lo que requirió utilizar comandos de sincronización e integración para unificar los cambios realizados por cada miembro.

Además, fue necesario coordinar la organización del código para evitar duplicaciones y asegurar que todas las funcionalidades funcionaran correctamente dentro de un único archivo principal.

## **8. CONCLUSIONES**

La realización de este proyecto permitió aplicar en un caso práctico los conocimientos adquiridos durante la cursada. A través de su desarrollo, se reforzó el uso de listas, diccionarios, funciones, archivos CSV y estructuras de control, integrando estos conceptos en una aplicación funcional.

Además, el trabajo brindó experiencia en el uso de herramientas de desarrollo colaborativo como Git y GitHub, favoreciendo la organización y el trabajo en equipo.

Por último, el proyecto permitió valorar la importancia de la modularización del código, la validación de datos y una adecuada gestión de la información, aspectos fundamentales para desarrollar programas más eficientes, escalables y fáciles de mantener.

## **9. BIBLIOGRAFIA Y WEBGRAFÍA**

*Python Software Foundation. Python Documentation. <https://docs.python.org>*

*Documentación Oficial de CSV. <https://docs.python.org/3/library/csv.html>*

*GitHub Documentation. <https://docs.github.com>*

*Material de estudio proporcionado por la cátedra.*

## **10. ENLACES DEL PROYECTO**

Repositorio GitHub: <https://github.com/Fran-spring/TPI-Prog1.git>

Link del video explicativo: <https://youtu.be/2fScaoshg7Y>